CHAPTER – III

CONTROL STRUCTURE AND LOOPING STATEMENTS

CONTROL STRUTURES:

Control structures are the decision making and looping statements in java. It checks the condition and executes the code accordingly.

TYPES:

- Simple If condition
- If else condition
- Else if ladder
- Nested if

SIMPLE IF CONDITION:

It is used to execute a block of code only when the condition is true.

SYNTAX:

```
if(condition) {
    //statements;
}
```

EXAMPLE:

```
public class Example{
    public static void main(String [] args){
        int age= 19;
        if(age>=18){
            System.out.println("Eligible to vote");
        }
    }   }
```

ELSE IF CONDITION:

It is used to check the condition if the condition is true, the true block will execute otherwise the else statement is executed.

SYNTAX:

```
if (condition) {
        //statements;
}
else {
        //statements;
}
```

EXAMPLE:

```
public class Example {
        public static void main(String[] args) {
                int age=18;
                if(age>=18){
                        System.out.println(“Eligible to vote”);
                }
                else {
                        System.out.println(“Not eligible”);
                }
```

ELSE IF LADDER:

The else if ladder is used to check more than one condition to execute a block of code.

SYNTAX:

```
if(condition) {
        //statements;
}
else if(condition) {
        //statements;
}
else if(condition) {
        //statements;
}
else {
        //statements;
}
```

EXAMPLE:

```
public class Example {
        public static void main(String [] args) {
                int marks = 50;
                if (marks>=40){
                        System.out.println(“Grade A”);
                else if (marks>=30){
                        System.out.println(“Grade B”);
                else if(marks>=20){
                        System.out.println(“Grade C”);
                else{
                        System.out.println(“Fail”);
                }
```

```
    }
}
```

NESTED IF CONDITION:

Nested if means a if statement contains another if statement in it.

SYNTAX:

```
if (condition1) {
    // executes if condition1 is true
  if (condition2) {
    // executes if both condition1 and condition2 are true
   }
 }
```

EXAMPLE:

```
public class NestedIfExample1 {
    public static void main(String[] args) {
        int age 20;
        String nationality "Indian";
        if (age 18) {
            if (nationality.equals("Indian")) (
                System.out.println("You eligible to vote in India.");
            } else {
            System.out.println("You are not eligible to vote in India.");
            }
        else {
```

```
                System.out.println("You must be at least 18 years old to vote.");
            }
        }
}
```

SWITCH CASE STATEMENTS:

Used to execute one block from many choices based on a variable's value.

SYNTAX:

```
switch (variable) {
        case value1:
            // statements
            break;
        case value2:
           // statements
           break;
        default:
          // statements
        }
```

EXAMPLE:

```
import java.util.*;
public class Calculator {
    public static void main(String[] args) {
        int a =10;
        int b=20;
        Scanner sc= new Scanner(System.in);
```

```
            System.out.println("1. Addition");
            System.out.println("2. Subtraction");
            System.out.println("3. Multiplication");
            System.out.println("4. Division");
            System.out.println("Enter your choice:");
            int choice= sc.nextInt();
            switch (choice) {
            case 1:
                  System.out.println("The Addition:"+(a+b));
                  break;
            case 2:
                  System.out.println("The Subtraction:"+(a-b));
                  break;
            case 3:
                  System.out.println("The Multiplication:"+(a*b));
                  break;
            case 4:
                  System.out.println("The Division:"+(a/b));
                  break;
            default:
                  System.out.println("Invalid choice!");
            }
      }

}
```

FOR LOOP:

Used when you know in advance how many times you want to repeat a block of code.

SYNTAX:

```
for (initialization; condition; update) {
   // statements
 }
```

EXAMPLE:

```
public class Example {
        public static void main(String [] args) {
                for(int i=1; i<=5; i++) {
                        System.out.println(“The number:”);
                }
        }
}
```

EXAMPLE 2 (DECREMENT LOOP):

```
 public class Example {
        public static void main(String [] args) {
                for(int i=1; i<=5; i++) {
                        System.out.println(“The number:”);
                }
        }
}
```

NESTED FOR LOOP:

The for loop contains another for loop in it.

EXAMPLE:

```
public class AlphabetPattern {
    public static void main(String[] args) {
        int n = 5;
        for (int i = 1; i <= n; i++) {
            char ch = 'A';
            for (int j = 1; j <= i; j++) {
                System.out.print(ch + " ");
                ch++;
            }
            System.out.println();
        }
    }
}
```

EXAMPLE 2 (PATTERN PRINTING):

```
public class InvPyr {
    public static void main(String[] args) {
        int n = 5;
        for (int i = n; i >= 1; i--) {
            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }
            for (int k = 1; k <= i; k++) {
```

```
                    System.out.print("* ");
                }
                System.out.println();
            }
        }
    }
```

EXAMPLE 3(HOLLOW PRINTING):

```
    public class InvRightTri {
        public static void main(String[] args) {
            int rows=5;
            int colu=5;
            for(int i=1; i<=rows; i++) {
                for(int j=1; j<=colu; j++) {
                    if(i==1||j==1||i+j==6)
                        System.out.print("*");
                    else
                        System.out.print(" ");
                }
                System.out.println();
            }
        }
    }
```

WHILE LOOP:

Used when you don’t know how many times to repeat — it runs as long as the condition is true.

SYNTAX:

```
while (condition) {
   // statements
 }
```

EXAMPLE:

```
public class EvenNum {
        public static void main(String[] args) {
                int i=2;
                while (i<=20) {
                        if(i%2==0)
                                System.out.println(i);
                        i++;
                }
        }
}
```

DO WHILE LOOP:

It is used to run the code at least ones even the condition is false.

SYNTAX:

```
do {
        // statements
 }
 while (condition);
```

EXAMPLE:

```
public class DoWhileExample {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println("Number: + i);
            i++;
        } while (i <= 5);
    }
}
```

ENHANCED FOR LOOP:

It is used to iterate through collections or arrays.

SYNTAX:

```
for (dataType variable : arrayName) {
    // statements
}
```

EXAMPLE:

```
public class EvenNumArr {
    public static void main(String[] args) {
        int [] arr= {1,2,4,5,8,6};
        int even=arr[0];
        int count=0;
        for(int num:arr) {
            if(num%2==0) {
                count++;
```

```
                }
            }
            System.out.println("the count:"+count);
        }
    }
```

BREAK STATEMENT:

The break statement is used to exit a loop or switch statement immediately, even before the condition becomes false.

EXAMPLE:

```
public class BreakExample1 {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                break
            }
            System.out.println(i);
        }
    }
}
```

CONTINUE STATEMENT:

The continue statement is used to skip the current iteration of a loop and move to the next iteration.

EXAMPLE:

```
public class ContinueExample1 {
    public static void main(String[] args) {
```

```
                for (int i = 1; i <= 10; i++) {
                        if (i == 5) {
                                continue; // skip iteration when i = 5
                        }
                        System.out.println(i);
                }
        }
}
```